Neural Networks: Final Project Report

Task 1: Fake News Detector (Section 1, Topic 11)
Task 2: Sequence Length Sensitivity in LSTMs (Section 2, Topic 21)

Group 3

June 2026

Abstract. This report presents Group 3's two final-project tasks for the Neural Networks course. Task 1 is an end-to-end fake-news classifier on the LIAR dataset, comparing a feature-based multilayer perceptron with an LSTM trained on raw text, and extended with a pretrained transformer and a metadata hybrid. We find that from-scratch models stall near an F1 of 0.53, that a pretrained DistilBERT reaching 0.60 shows the limit is knowledge rather than input, and that the speaker's track record predicts truthfulness better than the sentence itself. Task 2 is a controlled study of how an LSTM's accuracy and training time vary with input sequence length. Using a synthetic delayed-signal recall task we find that performance does not degrade smoothly but collapses at a memory horizon near thirty steps, that training time grows roughly linearly with length, and that the true cause is the distance between the cue and the readout rather than length itself. A vanilla recurrent network outperforms the default LSTM beyond the horizon; we trace this to the forget-gate bias initialization and show that setting it positive restores perfect accuracy at every tested length.

1. Introduction

The final project comprises two complementary tasks. Task 1 is an applied, end-to-end project whose goal is to build, train, and evaluate a working classifier. Task 2 is a focused, conceptual experiment whose goal is to explain why a particular phenomenon occurs. This report documents both, following the prescribed structure of problem, approach, experiments, analysis, and contribution. Both code notebooks execute from top to bottom with saved outputs and are fully reproducible.

2. Task 1: Fake News Detector (Section 1, Topic 11)

2.1 Problem

Fact-checking does not scale: there are far more claims being made than people to verify them. We ask whether a model can do a useful first pass, flagging a short political statement as likely fake or likely true so that a human reviewer can prioritise. We use the LIAR dataset (Wang, 2017), 12,836 PolitiFact statements each rated on a six-point scale from pants-fire to true. Following common practice we collapse this to a binary label: the three least-truthful ratings become FAKE and the three most-truthful become REAL, leaving the data mildly imbalanced at about 44 percent FAKE. We keep the dataset's official train / validation / test split of 10,269 / 1,284 / 1,283. Two things make the problem hard. The statements are short, a median of 17 words, so there is little text to reason about; and the binary mapping puts genuinely borderline claims right on the decision boundary, since a barely-true and a half-true are almost the same thing but receive opposite labels. We report precision and recall on the FAKE class throughout, because for a triage tool what matters is how many flagged claims are actually fake and how many of the fakes get caught.

2.2 Approach

Everything is built in PyTorch, and every neural model is trained with the same loop (Adam, binary cross-entropy loss, early stopping on validation F1) so that any difference comes from the model and not from how it was trained. All vectorizers, scalers, and vocabularies are fit on the training set only, and every run is seeded. The two required models are the core of the project. The MLP on hand-crafted features turns each statement into a TF-IDF vector (unigrams and bigrams, 5,000 terms) plus eleven linguistic features such as length, punctuation counts, sentiment, and lexical diversity, read by a small two-layer network. The BiLSTM on raw text tokenises the statement, learns word embeddings from scratch, and runs a bidirectional LSTM over the sequence. We keep the head-to-head fair: both models see only text features. LIAR also ships speaker metadata (party, and the speaker's past truthfulness record), which we hold out of the main comparison and study separately. We then add two models beyond the brief to make the findings more complete: a fine-tuned DistilBERT on the text, to test whether a pretrained model changes the picture, and a hybrid that concatenates the BiLSTM's text representation with the speaker metadata before a final classifier.

2.3 Experiments

Experiment 1: MLP versus BiLSTM on text. Averaged over five seeds the two models are tied. McNemar's test on their predictions gives p = 0.69, so the gap is noise, and both barely clear a logistic-regression baseline (F1 0.49). On text alone the sequence model's extra machinery buys nothing.

Model (text only)	Precision	Recall	F1	ROC-AUC
MLP (TF-IDF + linguistic)	0.53	0.50	0.514 (+/- 0.010)	0.62
BiLSTM	0.54	0.52	0.530 (+/- 0.018)	0.64

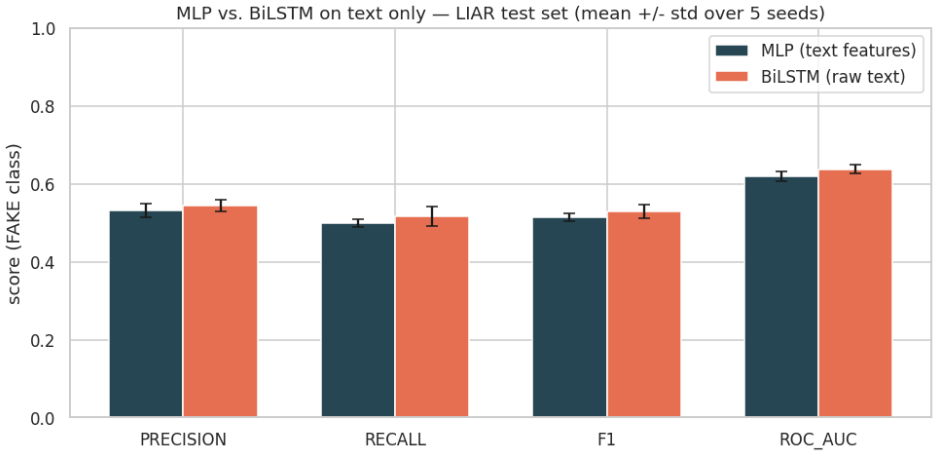

Figure 1. MLP versus BiLSTM on text only, mean and standard deviation over five seeds. The two architectures are statistically indistinguishable.

Experiment 2: where does the signal come from? Holding the MLP fixed and changing only its inputs, text features top out around F1 0.51. Adding speaker metadata lifts that to 0.56, and the result that surprised us most is that metadata on its own, just twelve numbers, scores F1 0.64 with an AUC of 0.75 and beats every text-based model. Most of what looks like fake-news detection on LIAR is really predicting whether this particular speaker tends to lie.

Experiment 3: LSTM design choices. A unidirectional LSTM slightly beat the bidirectional one (0.557 versus 0.520), hidden size barely mattered across 64, 128, and 256 units, and initialising the embeddings with pretrained GloVe vectors made no real difference (0.540 versus 0.544 for random). All three point the same way: the model is not held back by its capacity or its word vectors.

Experiment 4: threshold and class balance. Moving the decision threshold trades precision for recall as expected. We can push FAKE recall from 0.50 up to 0.95, but precision drops to 0.45 and the AUC does not move; the operating point is an application choice, not a genuine improvement.

Final comparison. Every model on the same test set, with bootstrap 95 percent confidence intervals on F1. DistilBERT comes out on top, but at 66 million parameters and about three minutes of training against roughly a million parameters and fifteen seconds for the others.

Model	Precision	Recall	F1	ROC-AUC	Accuracy	Params
LogReg (text)	0.57	0.43	0.49	0.65	0.61	5K
MLP (text)	0.56	0.47	0.51	0.65	0.61	1.3M
MLP (text + meta)	0.57	0.51	0.54	0.66	0.62	1.3M
BiLSTM (text)	0.53	0.56	0.55	0.64	0.60	1.2M
Hybrid (text + meta)	0.56	0.59	0.58	0.67	0.62	1.2M
DistilBERT (text)	0.63	0.57	0.60	0.69	0.67	66M

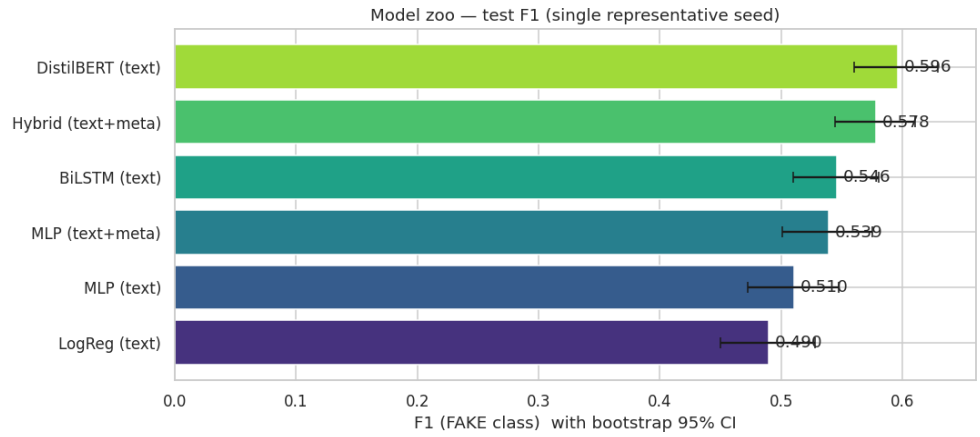

Figure 2. Test F1 by model with bootstrap 95 percent confidence intervals. Pretraining (DistilBERT) and metadata (Hybrid) give the only real gains.

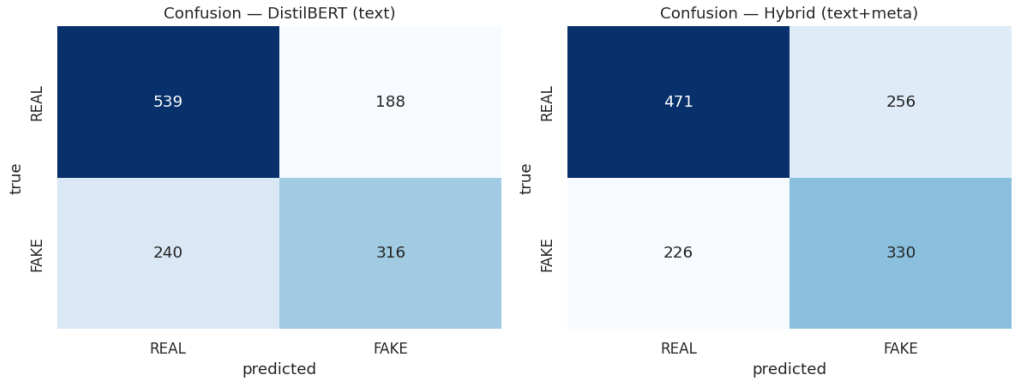

Figure 3. Confusion matrices for the two strongest models. Both under-detect the FAKE class, missing borderline statements rather than clear ones.

2.4 Analysis

The text is not the ceiling; knowledge is. Our from-scratch MLP and LSTM stall around F1 0.51 to 0.55, which tempts the conclusion that the words do not carry the answer. But DistilBERT, reading the exact same statements, reaches 0.60. The difference is not the input, it is what the model already knows: a network pretrained on a large corpus brings outside knowledge that ten thousand short examples cannot teach from scratch. Experiment 3 confirms this from the other side, since swapping in pretrained GloVe word vectors did nothing while a pretrained encoder helped. The useful knowledge lives in the whole encoder, not just the embeddings.

The speaker matters more than the sentence. Metadata on its own beats every text model on AUC. We are upfront about the catch: LIAR's credit-history counts include the current statement, so they partly leak how truthful the speaker usually is. That makes the metadata result as much a measure of reputation as of content, which is why we treat the clean text-only DistilBERT number as the honest headline. The practical point still holds, though, that who is speaking is a strong prior a real system would use.

The errors are consistent and informative. Every model under-detects FAKE, with recall below precision, and the mistakes cluster on the borderline labels: the MLP errs most on false and barely-true statements (error rates 0.52 to 0.57) and almost never on clear mostly-true ones (0.19). About a quarter of the test set is missed by both the MLP and the LSTM, a fair estimate of the floor, since these are claims that cannot be settled from eighteen words without checking the facts. An F1 near 0.6 looks modest, but for LIAR it is where the published work sits; numbers much higher usually mean the model is using leaked or external information, and we would rather report an honest figure with confidence intervals.

3. Task 2: Sequence Length Sensitivity in LSTMs (Section 2, Topic 21)

3.1 Problem

The assigned task is to train an LSTM on sequences of length 10, 50, and 200, measure accuracy and training time, and show how performance changes with length. Taken literally this is narrow, so we pose the deeper question a

recurrent network actually faces. Such a network must carry information from where it appears to where it is read out, one step at a time; the longer that path, the more the back-propagated gradient is attenuated, a difficulty known as the vanishing-gradient problem. The central question is therefore how far back an LSTM can reliably remember at a fixed training budget, and whether the limiting factor is the sequence length or the distance the information must travel.

3.2 Approach

We designed a synthetic delayed-signal recall task that isolates memory-over-distance from every other confound. Each input is a sequence of length L with two channels. Channel 0 contains independent N(0,1) noise at every step and acts as a distractor. Channel 1 is zero everywhere except at one known position, where it carries a clean marker: +1 for class 1 and -1 for class 0. The model reads the whole sequence and classifies from the final-timestep output. Because the marker is unambiguous wherever it appears, the only difficulty is remembering it across time; there is nothing to detect, and chance accuracy is exactly 0.50. The model is a single recurrent layer (either an LSTM or a vanilla RNN, each with 64 hidden units) followed by a linear head on the final-timestep output, so the only variable under test is the recurrent unit. Training uses Adam (learning rate 0.001), cross-entropy loss, a batch size of 64, and a fixed budget of 30 epochs for every run so that length comparisons are fair. Gradient-norm clipping at 1.0 is applied throughout; without it, occasional exploding-gradient steps destabilise even short-length training on unlucky seeds. Every configuration is repeated over 5 seeds, and we sweep the set 10, 20, 30, 50, 100, 200, exceeding the three required lengths to resolve the knee.

3.3 Experiments

Experiment 1: accuracy and training time versus length. With the marker at the start (the worst case, a distance of L minus 1 to the readout), accuracy is perfect up to L = 20, shows a sharp knee at L = 30, and sits at chance from L = 50 onward. Training time rises roughly linearly, as expected for a recurrent layer whose timesteps are processed sequentially.

L	10	20	30	50	100	200
Accuracy (mean)	1.000	1.000	0.902	0.498	0.486	0.502
Std. dev.	0.000	0.000	0.218	0.025	0.019	0.014
Train time (s)	5.1	6.8	6.5	11.3	21.8	38.9

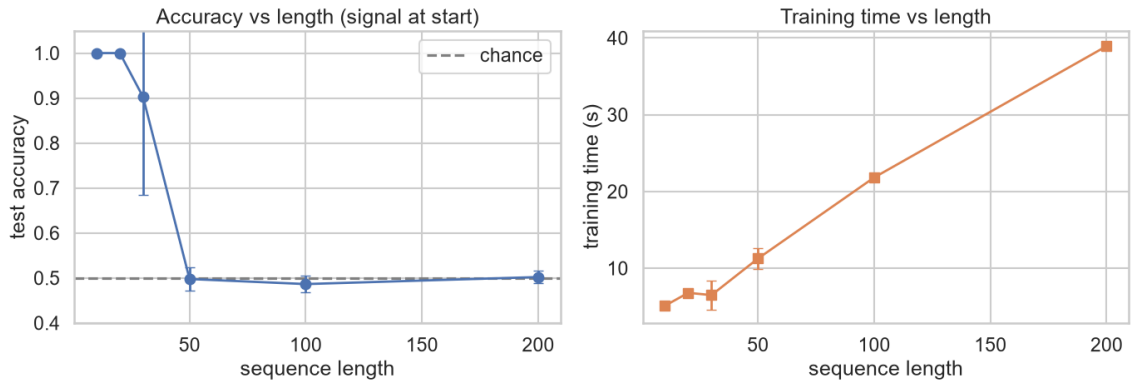

Figure 4. Test accuracy (left) and training time (right) versus sequence length. The error bar is widest at the L = 30 knee.

Experiment 2: distance, not length. We fix L = 200 for both conditions, so the computation is identical, and move only the marker: to the start (distance about 200) or to the end (distance about 0). If length were the cause, both would fail. Instead the marker-at-end condition is perfect while marker-at-start stays at chance. This is the headline result of the study.

Marker position (L = 200)	Accuracy	Std. dev.
Start (distance approx. 200)	0.502	0.014
End (distance approx. 0)	1.000	0.000

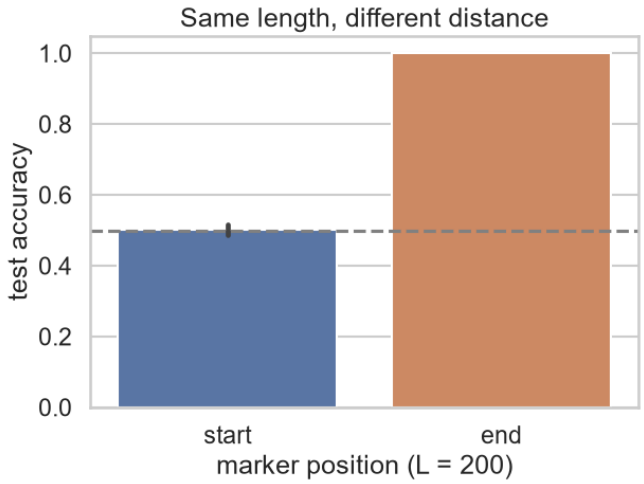

Figure 5. Identical length and computation; only the dependency distance differs.

Experiments 3 and 4: LSTM versus RNN, and the forget-gate fix. Contrary to the textbook expectation, beyond the knee a vanilla RNN out-remembers the default LSTM (at L = 100 the RNN reaches 0.90 against the LSTM's 0.49). The cause is initialization. PyTorch sets all LSTM biases to zero, so the forget gate begins at sigmoid(0) = 0.5 and the cell state is halved at every step, giving an effective horizon of only a few dozen steps. Re-initialising the forget-gate bias to +2, so that sigmoid(2) is about 0.88 and the cell retains about 88 percent of its state per step, restores perfect accuracy at every tested length with zero variance.

L	10	20	30	50	100	200
LSTM (default bias 0)	1.000	1.000	0.902	0.498	0.486	0.502
Vanilla RNN	1.000	1.000	1.000	0.898	0.896	0.606
LSTM (forget bias +2)	1.000	1.000	1.000	1.000	1.000	1.000

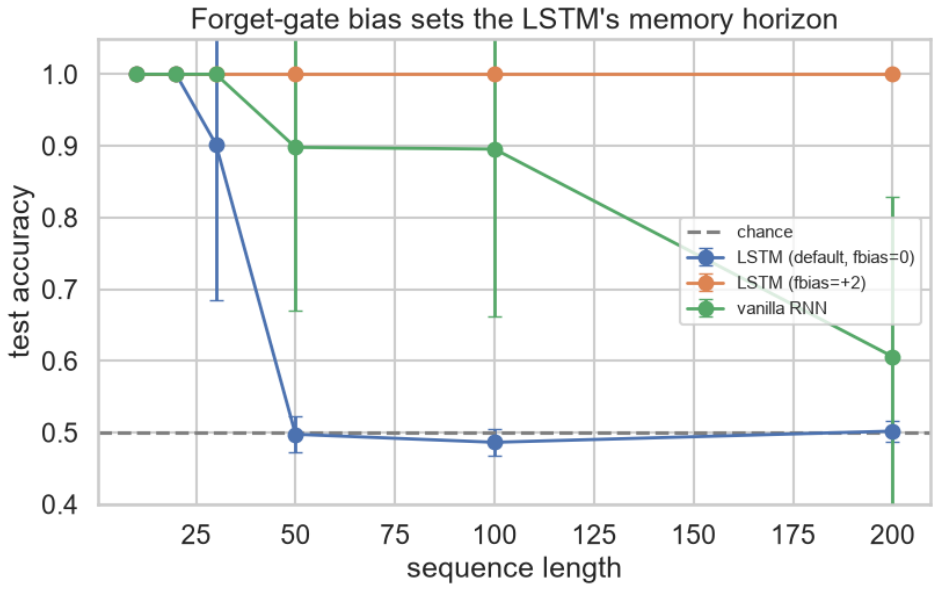

Figure 6. Mean accuracy versus length for the three models. The forget-gate bias, not the architecture, sets the LSTM's memory horizon.

3.4 Analysis

A memory horizon, not a smooth decay. Accuracy stays near 1.0 and then collapses at a knee around L = 30, where the error bar is widest: four seeds reach 1.0 while one remains at chance. This is the signature of an optimisation horizon under a fixed budget rather than a hard capacity wall. Training time is approximately linear in length, rising from about 5.1 seconds at L = 10 to about 38.9 seconds at L = 200, a factor of roughly 7.6 on CPU.

Distance, not length, is the cause. Experiment 2 is decisive: at identical length and computation, placing the dependency next to the readout restores perfect accuracy. What is colloquially called sequence-length sensitivity is in

fact sensitivity to dependency distance.

Gating helps only when it is initialised to remember. The LSTM cell update is c(t) = f(t) * c(t-1) + i(t) * g(t). With the default bias the forget gate f(t) is about 0.5 and leaks roughly half of the stored memory at every step; with a bias of +2 the gate is about 0.88 and the cell becomes a near-additive highway along which information and gradients survive hundreds of steps. The vanilla RNN, having no such built-in decay, learns a near-identity recurrence and so beats the mis-initialised LSTM. This connects directly to the vanishing-gradient analysis, in which repeated products of Jacobians with spectral norm below one decay geometrically in the number of steps. The clean marker channel removes signal detection as a confound, the fixed epoch budget makes the length comparison fair, and gradient clipping keeps the short-length baseline reliably perfect, so the horizon effect is unambiguous.

4. Reproducibility

Both notebooks run top to bottom with saved outputs. The Task 1 notebook (s1-p11-fake_news_detector.ipynb) downloads the data itself, uses the official splits, fits all preprocessing on the training set only, fixes all random seeds, and backs its comparisons with multi-seed averages, McNemar's test, and bootstrap confidence intervals. The Task 2 notebook (s2-p21.ipynb) uses a single configuration dictionary, a seed-setting helper, multi-seed runs, pandas result tables, and labelled figures; it was executed end to end on CPU using Python 3.12 and PyTorch 2.12.

5. Conclusion

The two tasks meet at the same lesson: architecture is rarely the binding constraint. In Task 1 the from-scratch MLP and LSTM are interchangeable, and the real gains come from outside knowledge (a pretrained encoder) and from who is speaking (metadata), not from a fancier text model. In Task 2 the LSTM's apparent length limit turns out to be an artifact of forget-gate initialization rather than of the architecture, and the deeper variable is dependency distance rather than raw length. The practical recommendations that follow are to invest in pretraining and informative features before deeper text models, to shorten dependency distance where the data layout permits, and to initialise the forget gate to remember before reaching for greater capacity, longer training, or attention-based models.

6. Contributions

Member	Primary Role	S1-P11 (Fake News)	S2-P21 (LSTM Length)
1	Data and EDA	LIAR pipeline, TF-IDF features, EDA	Synthetic data generator, reproducibility
2	MLP Baseline	MLP design, tuning, evaluation, error analysis	Vanilla RNN implementation and length sweep
3	BiLSTM and Hybrid	BiLSTM encoder, speaker-metadata hybrid	Core LSTM model, forget-gate bias ablation (Exp. 4)
4	DistilBERT and Eval	DistilBERT fine-tuning, calibration, cross-model analysis	Distance-control experiment (Exp. 2)
5	S2 Experiments	Figure formatting, report assistance	Length sweep (Exp. 1), full S2 analysis and notebook
6	Report and Slides	Report integration, slide deck, repo management	Report integration, slide deck, viva preparation